VICHARHUB SYSTEM ARCHITECTURE DOCUMENTATION

Liveblocks Architecture & Real-Time Collaboration in VicharHub

This document explains how **Liveblocks** works in VicharHub (Notion Clone), how it works in distributed web applications generally, the complete end-to-end data flow, and how real-time text editing, collaborative cursors, and live spreadsheets are implemented.

---

1. Executive Summary & Core Concepts

What is Liveblocks?

Liveblocks is a real-time infrastructure platform designed specifically for collaborative experiences (like Google Docs, Notion, Figma, and Miro). Instead of forcing developers to build and manage low-level WebSocket servers, presence caches, and conflict-resolution algorithms from scratch, Liveblocks provides a managed edge network with purpose-built SDKs.

In VicharHub, Liveblocks powers:

- **1. Multiplayer Document Editing:** Simultaneous rich-text typing using Yjs Conflict-Free Replicated Data Types (CRDTs) inside TipTap.
- **2. Collaborative Caret & Cursors:** Real-time visual indicators showing where other collaborators are typing, along with their names and colored avatars.
- **3. Presence Avatars:** Top-bar badges showing active users currently viewing a document or spreadsheet.
- **4. Real-Time Spreadsheet Sync:** Peer-to-peer event broadcasting for instant cell calculations across multiple windows.
- **5. Dynamic Identity Updates:** Instant propagation of name and color changes made in Settings to all peers across the room without page reloads.

---

2. System Architecture

The following diagram illustrates how VicharHub's clients, the Cloudflare Worker backend, the SQLite D1 database, and Liveblocks Cloud interact:

```
+--------------------------------------------------------------------------------+
```

VICHARHUB FRONTEND				
+------------------------------+ +------------------------+				
	Window 1 (User A: Anuj)		Window 2 (User B: Rohan)	
	- TipTap Editor (Yjs Doc)		- TipTap Editor (Yjs Doc)	
	- Liveblocks Presence		- Liveblocks Presence	
	- JSpreadsheet CE Grid		- JSpreadsheet CE Grid	
+---------------+---------------+ +-----------------+				

+-------------------+---+----------------------+

| |

1. Auth HTTP | 1. Auth HTTP |

Token Request | Token Request |

v v

+---+

CLOUDFLARE WORKER BACKEND (worker/index.ts)
Route: POST /api/liveblocks-auth
- Extracts user JWT from Bearer Header
- Verifies user in Cloudflare D1 Database (SELECT name FROM users WHERE id = ?)
- Contacts Liveblocks API with LIVEBLOCKS_SECRET_KEY
- Returns signed cryptographic capability token to client

+--+--+

|

| 2. Authorize Session

v

+---+

LIVEBLOCKS CLOUD EDGE NETWORK		
Room: page-{pageId}		
+---------- ----------+		
	WebSocket Gateway	
	- Yjs CRDT Synchronization Channel (TipTap document updates)	
	- Awareness / Presence Channel (Carets, selections, display names)	
	- Custom Broadcast Bus (SHEET_CELL_CHANGE events)	
+---------- ----------+		

+----------+----------+

|

| 3. WebSocket Real-Time Sync (16ms throttle)

v

+----------+

PERSISTENCE & STORAGE TIER
- Instant Local Paint: localStorage (vicharhub-cache via Zustand persist)
- Durable Cloud Storage: Cloudflare D1 SQLite (pages table) via debounced PATCH
- Flush on Unload: Native browser keepalive: true requests on reload / exit

+----------+

---

3. The Four Core Primitives of Liveblocks

Liveblocks provides four fundamental real-time primitives. VicharHub utilizes three of them:

Primitive	What It Does	Where Used in VicharHub	Transport
Authentication	Authorizes client sessions using HMAC signed tokens.	worker/index.ts (handleLiveblocksAuth)	HTTPS POST
Presence	Ephemeral, in-memory state representing temporary user data (cursors, selection, names, viewing status). Automatically destroyed on disconnect.	PresenceAvatars.tsx, TipTap Caret extension, useMyPresence, useOthers	WebSocket
CRDT Collaboration (Yjs)	Conflict-free text merges for rich-text editing where simultaneous typing from multiple users must never lose characters or conflict.	NotionEditor.tsx via @liveblocks/react-tiptap	WebSocket
Broadcast Events	Fire-and-forget peer messaging without persisting to permanent room storage. High speed, minimal overhead.	SpreadsheetEditor.tsx via useBroadcastEvent / useEventListener	WebSocket

---

4. Deep Dive: How Each Component Works in VicharHub

A. Authentication & Room Authorization (src/lib/liveblocks.ts & worker/index.ts)

Every Liveblocks connection requires a signed authorization token from your backend server. This ensures that unauthorized users cannot eavesdrop on rooms.

1. Client Requests Room Access (src/lib/liveblocks.ts)

When a document or spreadsheet mounts, Liveblocks triggers the client's uthEndpoint:

ypescript

```
const client = createClient({
throttle: 16, // 60 updates/sec maximum to preserve bandwidth
authEndpoint: async (room) => {
const accessToken = localStorage.getItem('vh_access_token') ?? '';
const response = await fetch('/api/liveblocks-auth', {
method: 'POST',
headers: {
'Content-Type': 'application/json',
Authorization: Bearer ,
},
body: JSON.stringify({ room }),
});
return response.json();
```

```
},
});
```

2. Worker Validates User & Signs Token (worker/index.ts)

In the Cloudflare Worker:

- **1.** The server inspects the Authorization: Bearer header and identifies the user.
- **2.** It queries Cloudflare D1 for the user's latest name (SELECT name FROM users WHERE id = ?).
- **3.** It makes a server-to-server call to Liveblocks:

```
ypescript
const response = await fetch('https://api.liveblocks.io/v2/authorize-user', {
method: 'POST',
headers: {
Authorization: Bearer ,
'Content-Type': 'application/json',
},
body: JSON.stringify({
userId,
userInfo: { name: userName, color: userColor },
permissions: { '*': ['room:write'] },
}),
});
```

- **4.** Liveblocks returns a signed token, and the client establishes a persistent WebSocket connection to the room: page-.

B. Document Collaboration: TipTap + Yjs CRDTs (NotionEditor.tsx)

Traditional databases or WebSockets cannot easily handle two users typing in the exact same paragraph simultaneously (one user's text will overwrite the other's).

VicharHub solves this using **CRDTs (Conflict-free Replicated Data Types)** powered by **Yjs** and bridged by @liveblocks/react-tiptap.

How Yjs CRDT Works

- Instead of treating text as a flat string (e.g., 'Hello world'), Yjs treats the document as an ordered tree of unique characters, each with an immutable unique ID (Client ID + Lamport clock timestamp).

- When User A types 'A' between character 2 and 3, and User B simultaneously types 'B' at the same spot:

- Both operations are assigned distinct deterministic orders based on their IDs.

- When the operations cross each other over WebSockets, Yjs merges them deterministically into either 'AB' or 'BA' across all connected clients.

- **No characters are lost, and no server locks are required.**

Wiring TipTap to Liveblocks

In NotionEditor.tsx:

```
ypescript
const liveblocks = useLiveblocksExtension({
initialContent: isSpreadsheet ? '
' : page?.content || '
',
});
const editor = useEditor({
extensions: [
liveblocks, // Bridges TipTap transactions to Liveblocks Yjs WebSocket
// ... StarterKit, SlashCommand, CodeBlock, Tables
],
});
```

Collaborative Caret Awareness

TipTap's collaboration extension includes a caret provider. To ensure that collaborators immediately see updated display names:

```
ypescript
// Dynamically updates the local user awareness in Yjs
(editor.commands as any).updateUser({ name, color });
```

When User A moves their cursor, their caret position and current name are broadcast over the Yjs awareness channel. User B's screen renders a colored vertical line with User A's name tag floating directly above their cursor.

C. Spreadsheet Collaboration: Event Broadcast (SpreadsheetEditor.tsx)

Unlike rich text, spreadsheets consist of independent grid cells (Row Y, Col X). Using heavy CRDT document trees for high-frequency cell edits would be unnecessarily resource-heavy.

Instead, VicharHub utilizes **Liveblocks Custom Broadcast Events**:

1. Broadcasting Cell Edits

Whenever a cell is modified in JSpreadsheet (onchange or onafterchanges):

```
ypescript

broadcastRef.current({
type: 'SHEET_CELL_CHANGE',
sheetIndex: activeSheetIndexRef.current,
x: Number(change.x),
y: Number(change.y),
value: change.value,
});
```

2. Listening and Applying Remote Edits

All other clients subscribed to the same room listen with useEventListener:

```
ypescript

useEventListener(({ event }) => {
if (event.type === 'SHEET_CELL_CHANGE') {
// 1. Update in-memory sheets data
updatedSheets[event.sheetIndex].data[event.y][event.x] = event.value;
// 2. If viewing this tab, update the JSpreadsheet DOM grid without re-rendering
worksheet.setValueFromCoords(event.x, event.y, event.value, true);
}
});
```

This guarantees **instant calculations and multi-user cell sync** with virtually zero latency.

D. Presence & Avatars (PresenceAvatars.tsx)

The presence avatars in the top-right corner show everyone currently in the room:

```
ypescript
export default function PresenceAvatars() {
const others = useOthers(); // Live array of all other connected clients
const currentUser = useSelf(); // The current user's room state
return (
{others.map((other) => {
// Reads dynamic presence name or fallback info name
const name = other.presence?.displayName || other.info?.name || 'Anonymous';
const color = other.presence?.user?.color || other.info?.color || '#4f46e5';
return ;
})}
);
}
```

Whenever a user opens the tab, Liveblocks adds them to others. When they close the tab, Liveblocks automatically fires a disconnect event and removes their avatar within 100 milliseconds.

E. The Hybrid Persistence Model: Live vs Durable Storage

A common question is: *Does Liveblocks replace the database?*

No. VicharHub uses a Hybrid Architecture:

```
+-----------------------------------------------------------+
| REAL-TIME TIER (Liveblocks) | | - Millisecond updates for active users | | - Carets, cursors, avatars | |
- Ephemeral in-flight changes |
+----------------------------+----------------------------+
|
| Debounced 600ms / Blur / Unload
v
+-----------------------------------------------------------+
| DURABLE TIER (Cloudflare D1) | | - Long-term permanent storage (pages table) | | - Offline cache
(localStorage instant paint) | | - Survives browser restarts, server resets, and reboots |
+-----------------------------------------------------------+
```

- **1. Typing / Editing:** Liveblocks handles live synchronization between connected users in real time.
- **2. Debounced Cloud Save:** In usePageStore.ts, an auto-save timer waits 600ms of inactivity and sends a PATCH /api/pages/:id with the latest document HTML or spreadsheet JSON to Cloudflare D1.
- **3. Emergency Unload Flush:** On page refresh (F5) or tab close, eforeunload captures any active cell editor, commits its value, and dispatches an asynchronous HTTP request using

etch(..., { keepalive: true }).

---

5. End-to-End Workflow Scenarios

Scenario 1: A User Types in a Document

- **1.** User A types 'Hello World' in NotionEditor.tsx.
- **2.** TipTap dispatches a transaction to the @liveblocks/react-tiptap extension.
- **3.** The extension updates the local Yjs document and streams a binary CRDT update chunk over WebSocket to Liveblocks Cloud.
- **4.** Liveblocks broadcasts the chunk to User B in the same room (page-{pageId}).
- **5.** User B's Yjs provider receives the chunk, merges it into User B's local document tree, and TipTap automatically re-renders the text in User B's window.
- **6.** User A's client pauses for 600ms, then sends PATCH /api/pages/:id to Cloudflare Worker to permanently store the updated HTML in D1.

Scenario 2: A User Updates Their Name in Settings

- **1.** User A opens Settings and changes their name to 'Anuj'.
- **2.** SettingsPanel.tsx updates useAuthStore and dispatches COLLAB_NAME_CHANGE_EVENT.
- **3.** PresenceNameSync in CollaborativeEditor.tsx catches the event and calls:

```
ypescript
updateMyPresence({ displayName: 'Anuj', user: { name: 'Anuj' } });
```

- **4.** NotionEditor.tsx catches the event and updates TipTap Yjs awareness:

```
ypescript
editor.commands.updateUser({ name: 'Anuj' });
```

- **5.**

oom.reconnect() initiates a quick WebSocket reconnection.

- **6.** In User B's window:

- The avatar in PresenceAvatars.tsx updates to 'Anuj'.

- The floating tooltip above User A's cursor updates to 'Anuj'.

---

6. Summary of Key Files

File	Purpose
src/lib/liveblocks.ts	Initializes Liveblocks client, typed Presence schema, room context, and hooks.
worker/index.ts	Cloudflare Worker endpoint (/api/liveblocks-auth) that signs authorization tokens.
src/components/editor/CollaborativeEditor.tsx	Scopes Liveblocks RoomProvider to page- and syncs presence identity.
src/components/editor/NotionEditor.tsx	TipTap collaborative text editor with Yjs CRDT binding and caret awareness.
src/components/editor/SpreadsheetEditor.tsx	Collaborative spreadsheet using Liveblocks broadcast events and robust JSON persistence.
src/components/editor/PresenceAvatars.tsx	Live avatars rendering active collaborators in the current room.